

Terraform vs. AWS S3 Lifecycle Eventual Consistency

Timeout Issue Root Cause Analysis & Optimization

Ruiling Jin, Kaiyue Wei

Introduction

This project investigates the root cause of the notorious Terraform S3 lifecycle timeout (GitHub #25939), hypothesizing that a conflict between Terraform's expectations and AWS's eventual consistency model control-plane causes intermittent timeout issue.

Methodology

- We empirically validated our hypothesis by developing a custom Python benchmarking suite that isolates and measures the distinct consistency behaviors of the S3 Data Plane versus the Control Plane.
- We simulated realistic S3 propagation delay using percentile-based ranges and tested 4 waiter strategies (Baseline, Extended, Adaptive Learning and Hybrid Dense-Sparse Polling), each with 30 iterations.

Environment Limitations and Simulation of Real-World Delays:

- AWS Learner Lab cannot reproduce Production timeout issues due to its small, low-latency environment (<1s)
- Production AWS involves large, multi-data-center systems where configuration changes propagate across many nodes with heavy-tailed delays
- Our Solution: Simulated real-world S3 propagation delays by assigning random delays based on production percentiles
- This approach recreates long-tail propagation behavior, enabling Learner Lab to emulate production-like conditions

Literature cited

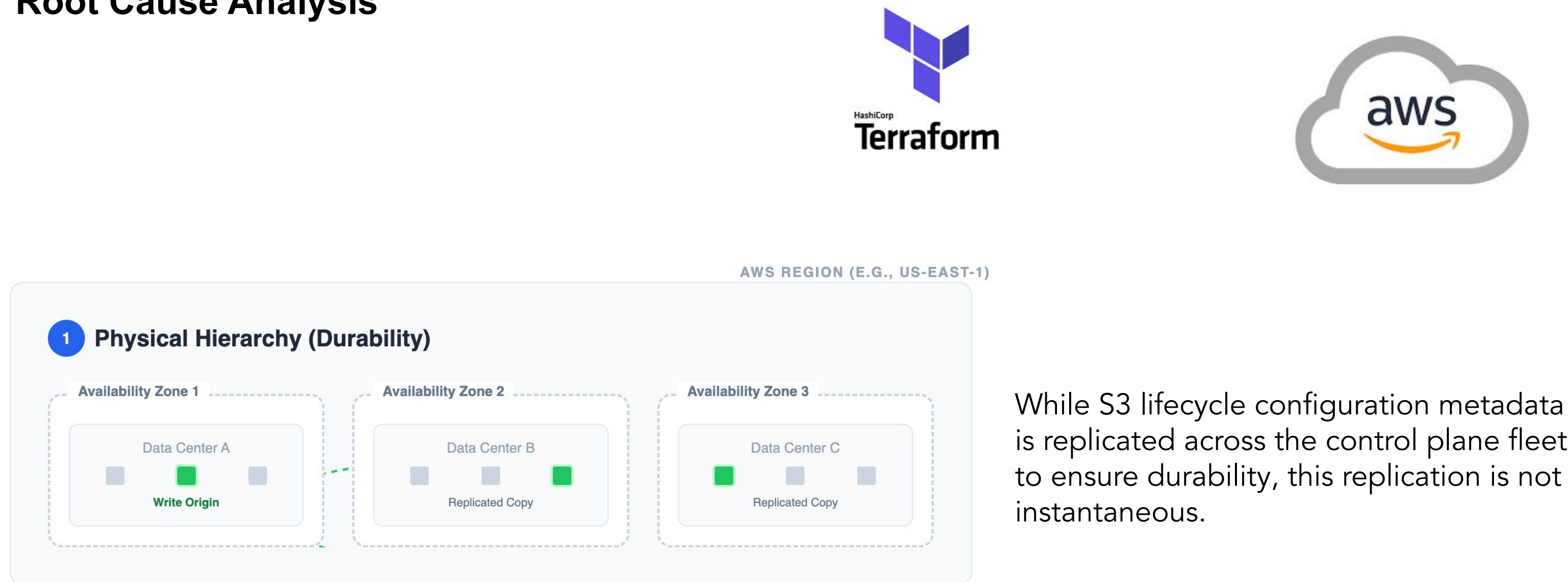
[1] Amazon Web Services. Amazon S3 API Reference - Lifecycle Configuration. AWS Documentation, 2024.

[2] HashiCorp. Terraform AWS Provider - Issue #25939: Lifecycle configuration for S3 Bucket failing with timeout.

[3] Terry, D. B., et al. 2011. Replicated data consistency explained through baseball. *Microsoft Research*.

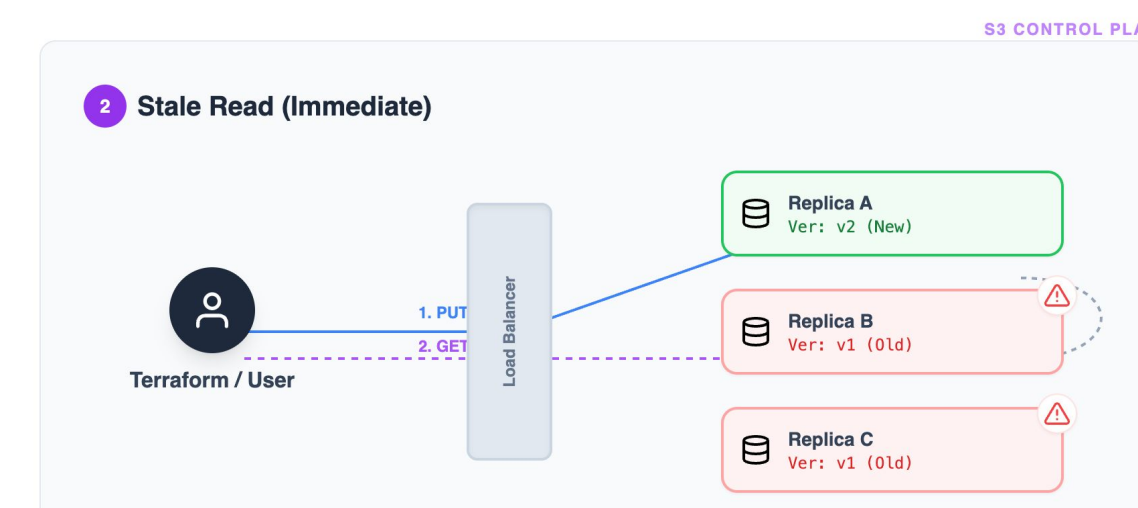
[4] Vogels, W. 2021. Diving deep on S3 consistency. *All Things Distributed*.

Root Cause Analysis



This propagation delay introduces two distinct challenges for infrastructure-as-code tools like Terraform.

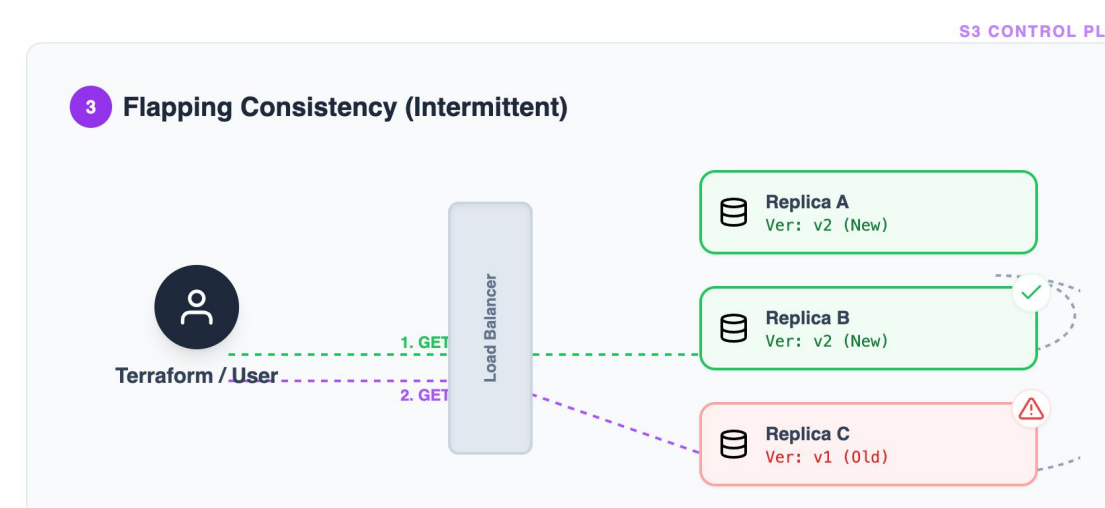
Challenge 1: Stale Reads



How Terraform handles?

Relies on a fixed polling interval (a "magic number," 5 seconds).

Challenge 2: Flapping Consistency



How Terraform handles?

It requires the success state to be observed consistently across consecutive checks (10 times in a row)

The fundamental flaw in this waiter logic is that it is inefficient for the "happy path" (wasting time verification when the system is already consistent) yet too rigid for the "tail latency path" (failing to adapt when AWS propagation is slower than usual).

This rigidity is the primary driver behind the s3_lifecycle_configuration timeout errors.

Conclusions

- This outcome highlights a critical friction point in cloud engineering: How Terraform handles S3's eventual consistency. By empirically mapping the "stale read" windows of the control plane, we demonstrated that standard timeout logic is mathematically destined to fail against the statistical distribution of metadata propagation.
- Our solution effectively "bridges" this consistency gap, proving that client-side adaptability—not brute-force waiting—is the only way to reliably manage asynchronous distributed systems.

Further information

Please see <https://github.com/KaiyueWei/s3-consistency> for more details.

We are at ranaruiling@gmail.com and kaiyue.wei@outlook.com if you have a question or comment.

Optimization

We evaluated four polling strategies under simulated production delays.

- Baseline** uses fixed short-interval polling and a standard timeout, reproducing some real-world failures.
- Extended Timeout** keeps fixed intervals but extends the timeout to eliminate failures, at the cost of many unnecessary API calls.
- Adaptive Learning** adjusts polling intervals and timeouts dynamically based on past results, improving success but with variable efficiency.
- Hybrid Polling** applies a fixed dense-early, sparse-late schedule, capturing common cases quickly while reducing unnecessary calls in rare slow cases, achieving predictable full coverage with the best reliability-to-efficiency balance.

By contrast,

- Hybrid's fixed dense-early / sparse-late design closely matches the heavy-tailed S3 propagation distribution: frequent checks during common P50–P90 cases and sparse checks for rare tail cases.
- This ensured full P99 coverage, predictable behavior, and significantly fewer unnecessary API calls, making Hybrid the most reliable, stable, and cost-efficient strategy for Terraform's S3 lifecycle waiter.

